

INTERNSHIP ASSIGNMENT REPORT

By: E Jayanth

Data

To test these models properly, I knew that quiet, indoor audio recordings would not give useful results. Because I had to travel outside, I used the opportunity to record all 21 audio samples directly on the road in live traffic. This provided a genuinely messy, real-world dataset of Hinglish and Kannada mixed with horns, loud music.

In total there are 21 audio samples. 12 with real life traffic and 9 with clear and calm background noise (with only fan ON in the background).

I did not use the optional open-source datasets because they were mostly formal recordings like “Mann ki Baath” which did not represent the real-world use cases that Vahan needed.

Models

I selected three models to benchmark, focusing on the trade-offs between cost, speed, and real-world system design:

- **Deepgram (Nova-3):** The one mentioned to be used compulsorily in the assignment and also a commercial standard.
- **Sarvam (Saaras):** A localized model that had made headlines recently and was built specifically for Indian languages.
- **Whisper V3 (using Groq):** Whisper is usually too slow for real-time use, but Groq's hardware accelerates it heavily. I wanted to see if we could get open-source quality at high speeds.

Quantitative Results

Here is how they performed on successfully extracting the correct Bangalore location names:

Model	Average Latency	Location Match Rate (ERR)	Average Word Error Rate (WER)	Output Language Script
Deepgram	2.68 seconds	23.8% (Caught 5 of 21)	81.8%	English (Latin)
Whisper V3 (via Groq)	1.30 seconds	14.3% (Caught 3 of 21)	99.1%	Mixed / Kannada
Sarvam	1.43 seconds	0.0% (Caught 0 of 21)	105.1%	Hindi (Devanagari)

The high error rates were a concern. The transcripts were mostly accurate but in Indian languages itself (like Hindi/Kannada). This meant when comparing a model's native Indic text output against a

ground truth with English text, it artificially inflates the error rate despite the model hearing the audio perfectly. Eg, there 0% word-overlap between 'Bellandur' and 'बेलंदूर' which are actually the same name written in different languages.

Evaluation Metrics

I measured two different types of error rates to get a complete picture of model performance:

- **Word Error Rate (WER):** It calculated errors by counting every single Substitution, Deletion, and Insertion the model makes compared to the answer key.
 - The average WER for Sarvam exceed 100%. This was because the math penalizes every extra word (Insertions). If the ground truth is one word ("HSR"), but the model splits and outputs them as three words ("एच", "एस", "आर"), the algorithm counts 2 massive insertion errors. This pushes the error rate far past 100%, even if the model actually heard the audio correctly.
- **Location Match Rate / Entity Recovery Rate (ERR):** This is the most critical business metric. It measures a strict Pass/Fail: *Did the model successfully catch the specific Bangalore locality name?* If the ground truth was "Bellandur", and the model successfully printed "Bellandur" (or a very close phonetic spelling like "Belindur"), it passed. If it missed the location entirely, it failed.

Further analysis identified the following root causes for the low scores even though the models performed decent:

1. The difference in the language in which the output is compare to the ground truth. I had written the ground truth in English: "Aap kahan ho sir?". But the models gave in Indic scripts which made comparing them useless because of the difference in language used to write.
2. Deepgram did something known as transliteration, where it tried to give out what it heard in English (or Hinglish) like: "By a auto yarns Janagar 4 Block". But this was also very messed up because the above example was actually "Bhaiya auto yahan se Jayanagar fourth block jayega kya?"
3. AI models type what a word *sounds* like, not necessarily how it is spelled on a map. For example, Deepgram correctly heard a location but spelled it as "Belindur" instead of the official spelling "Bellandur".

Effect of Background Noise

There were 12 audio samples which I recorded on the streets near busy roads. One even had a loud music being played in the background. But my voice was clear and could be heard. The models performed differently here:

1. Example 1:
 - Ground Truth: "Haan mai Koramangala signal pe khada hoon, traffic bohot hai time lagega sir."
 - Deepgram: "Signal traffic time" (Completely erased "Koramangala")
 - Sarvam: "हां, मैं अभी कोरमंगला सिग्नल पे खड़ा हूँ..." (Perfect)

2. Example 2:
 - Ground Truth: "Hello? Main Peenya industrial area mein hoon, aap kahan ho sir?"
 - Deepgram: "Hello" (Dropped everything else)
- **These showed Deepgram's noise cancellation algorithms aggressively filtered out the voice entirely, assuming it was background noise.**
3. Example 3:
 - Ground Truth: "Hello sir. Abhi Hebbal flyover ke neeche hun sir..."
 - Sarvam Output: "हेलो सर, अभी एयरपोर्ट फ्लाईओवर के नीचे हूं..."
 - Whisper: "हेलो सर अभी एप फ्लाईवर के नीचे हुए सर आवाज आ रहा है हेलो"
4. Example 4:
 - Ground Truth: "Mai abhi JP Nagar ninth phase wale layout mein hoon."
 - Deepgram Output: "May be j p number nine three layout" (Heard "number" instead of "nagar", and "three" instead of "phase")
- **This showed that the models did try to transcribe through the traffic, but the noise muffled the voice and caused hallucinations.**

Conclusion

So, in short, for loud noises in the background, the models performed differently:

- Deepgram had issues because of its aggressive noise filtrations. If the traffic is too loud, it deletes the audio, resulting in missing entities
- Sarvam pushes through the noise and is incredibly accurate (Shivaji nagar, Koramangala), but outputs Devanagari script, which breaks the backend string-matching.
- Whisper V3 is prone to severe hallucinations (छ. छ. छ.) when loud noise mixes with language-switching.

But as you see, the models could listen decently well but scored less because of limitations and discrepancies in text. So, a few recommended architectural improvements:

1. A transliteration pipeline which could write in English which would reduce the WER drastically.
2. Better spelling of the names (sticking to the official names of the places). Maybe pass custom vocabulary to the models to give it a hint about what words to look for.
3. Better background noise filtration. Deepgram's one was too aggressive and Whisper's one was also messy. Sarvam's performed well but not perfect. It caused hallucinations in the models. Maybe a Voice Activity Detection (VAD) and also better noise filtering algorithms would help the models perform better.